

OPERATOR GUIDE

# Platform Administrator Manual

Deploying, onboarding, and maintaining the attendance platform

## WHO IT'S FOR

The platform administrator — the developer or operator who runs the service

## VERSION

Version 3.0 | September 2026 |  
Firmware 1.10.0

## IN THIS GUIDE

- 1 Introduction
- 2 How the system fits together
- 3 Initial deployment
- 4 Firmware
- 5 Onboarding an institution
- 6 Secrets and scanner security
- 7 Over-the-air releases
- 8 Database
- 9 Troubleshooting
- 10 Pre-handover checklist

## THE DOCUMENTATION SET

Platform admin

Super admin

Admin

Staff

School

Office

Shop

Privacy note

Technical

# Contents

|                                        |          |
|----------------------------------------|----------|
| <b>1. Introduction</b>                 | <b>2</b> |
| 1.1 Roles at a glance                  | 2        |
| 1.2 Institution types                  | 2        |
| <b>2. How the system fits together</b> | <b>2</b> |
| 2.1 How scanners authenticate          | 2        |
| <b>3. Initial deployment</b>           | <b>3</b> |
| 3.1 Supabase                           | 3        |
| 3.2 Vercel                             | 4        |
| 3.3 Your platform admin account        | 4        |
| <b>4. Firmware</b>                     | <b>4</b> |
| 4.1 Prerequisites                      | 4        |
| 4.2 secrets.h                          | 4        |
| 4.3 certs.h                            | 5        |
| 4.4 Flashing                           | 5        |
| <b>5. Onboarding an institution</b>    | <b>5</b> |
| 5.1 Creating it                        | 5        |
| 5.2 Attaching scanners                 | 6        |
| 5.3 What to hand each client           | 6        |
| 5.4 Creating a super admin by hand     | 6        |
| 5.5 Institution lifecycle              | 6        |
| <b>6. Secrets and scanner security</b> | <b>7</b> |
| 6.1 Per-scanner secrets                | 7        |
| 6.2 BOOTSTRAP_SECRET                   | 7        |
| 6.3 CRON_SECRET                        | 7        |
| <b>7. Over-the-air releases</b>        | <b>7</b> |
| <b>8. Database</b>                     | <b>8</b> |
| <b>9. Troubleshooting</b>              | <b>9</b> |
| <b>10. Pre-handover checklist</b>      | <b>9</b> |

## 1 Introduction

This manual is for whoever runs the platform: one cloud deployment that serves every institution. You deploy the backend and dashboard, build and flash scanners, onboard institutions, and keep secrets and certificates current.

`platform_admin` is the only role that sees every institution. Protect its credentials accordingly. For the design behind what follows — schema, edge-function contracts, firmware internals — see the **Technical Documentation** alongside this manual.

### 1.1 Roles at a glance

| Role                                      | Scope                              | Created by                                 |
|-------------------------------------------|------------------------------------|--------------------------------------------|
| <code>platform_admin</code>               | Every institution                  | You, in SQL (section 3.3)                  |
| <code>super_admin</code>                  | One institution, full control      | You at onboarding; then other super admins |
| <code>admin</code>                        | One institution, or one unit of it | A super admin                              |
| <code>teacher</code> / <code>staff</code> | One unit, read-only                | A super admin                              |
| <code>cashier</code>                      | One shop: clients, sales, catalog  | A super admin                              |

### 1.2 Institution types

| Type                | Adds                                                        | Hides                                         |
|---------------------|-------------------------------------------------------------|-----------------------------------------------|
| <code>school</code> | Academic terms, year-end promotion                          | —                                             |
| <code>office</code> | Periods and holidays                                        | Promotion                                     |
| <code>shop</code>   | Clients, sales, catalog, loyalty, reports, the cashier role | Promotion, academic terms, the student roster |

## 2 How the system fits together

Three tiers, all yours:

- **ESP32 firmware** — one Arduino sketch, flashed over USB once and updated over the air after that
- **Supabase** — PostgreSQL 17, six Deno edge functions, `pg_cron`, and `pg_net`
- **Next.js dashboard** — on Vercel; every Supabase call runs server-side, so no key ever reaches a browser

All traffic is HTTPS. Scanners validate server certificates against a CA bundle compiled into the firmware (section 4.3).

### 2.1 How scanners authenticate

Each scanner holds its **own** secret, issued when it is first assigned and stored in `devices.device_secret`. A compromised scanner exposes only itself. `devices.revoked = true` is the server-side kill switch: the edge functions refuse that scanner immediately, whether or not the physical unit is ever wiped.

**NOTE**

Older builds shared one `institutions.device_secret` per institution. That column survives only as a transitional fallback until every scanner has re-provisioned — see `docs/for-operators/device-secret-migration-rollout.md`.

**3**

## Initial deployment

### 3.1 Supabase

- 1 Create a Supabase project.
- 2 Enable the **pg\_net** and **pg\_cron** extensions (Database → Extensions). The nightly absence job can't call `net.http_post()` without `pg_net`.
- 3 Store the two values the nightly job reads from Vault at run time:

```
select vault.create_secret('https://<ref>.supabase.co', 'supabase_project_url');
select vault.create_secret('<a long random string>', 'cron_secret');
```

SQL

- 4 Run every migration in `backend/supabase/migrations/`, in filename order. The last cron migration schedules `mark-absent-daily` at **21:00 UTC**.
- 5 Deploy the edge functions and set their secrets. `CRON_SECRET` must equal the Vault `cron_secret`; `BOOTSTRAP_SECRET` must equal the one in the firmware's `secrets.h`.

```
cd backend/supabase
supabase functions deploy --project-ref <ref>
supabase secrets set --project-ref <ref> BOOTSTRAP_SECRET=<value> CRON_SECRET=<value>
```

BASH

`SUPABASE_URL` and `SUPABASE_SERVICE_ROLE_KEY` are provided to functions automatically.

**IMPORTANT**

Migrations carry a "do not apply to cloud blind" header for a reason. Read each one before applying — several widen a constraint that must land before the matching dashboard deploy, and a few are order-sensitive.

**NOTE**

`mark-absent` works out "today" in each institution's own timezone, so it must run after the last scan of the day and before local midnight everywhere you operate. 21:00 UTC suits West Africa and Europe; reschedule in a new migration if you onboard institutions far from UTC.

## 3.2 Vercel

- 1 Import the repository into Vercel with `frontend` as the root directory, and confirm which branch Production deploys from.
- 2 Set exactly three environment variables: `NEXT_PUBLIC_SUPABASE_URL` , `NEXT_PUBLIC_SUPABASE_ANON_KEY` , and `SUPABASE_SERVICE_ROLE_KEY` .
- 3 Deploy.

## 3.3 Your platform admin account

Create your user in Supabase Auth, copy its UUID, and give it the platform role:

```
insert into profiles (id, role) values ('<your-auth-user-uuid>', 'platform_admin');
```

SQL

A platform admin has no institution. Signed in, you see **Institutions** and **Create institution** in the menu, and every page can be filtered by institution.

# 4 Firmware

## 4.1 Prerequisites

- Arduino IDE 2.x with the Espressif **ESP32** board package
- Libraries: **Adafruit Fingerprint Sensor Library**, **RTCLib**, **Adafruit GFX Library**, **Adafruit SSD1306**, and **ArduinoJson** (version 7)
- WiFi, HTTPS, the web server, DNS, SPIFFS, and OTA updates come with the ESP32 core — nothing extra to install
- A USB cable

## 4.2 secrets.h

Copy `firmware/ClassAttendance_Current_RTC/secrets.example.h` to `secrets.h` (gitignored). It defines one value, which must match the edge functions' `BOOTSTRAP_SECRET` :

```
#define BOOTSTRAP_SECRET "your_bootstrap_secret_here"
```

C

**There is no compile-time WiFi.** A factory-fresh scanner broadcasts `Attendance-Setup` (password `setup1234` ) with a setup page at `192.168.4.1` , and whoever is on site enters the real network. You never need a client's WiFi password to build their scanner. After that first boot, only an enrolled master finger can reopen the portal.



### IMPORTANT

Enrol a master finger on every scanner before it leaves your hands, and make sure the client enrolls their own. Without one, a WiFi change means the scanner comes back to you for reflashing — the most common avoidable support call.

## 4.3 certs.h

`certs.h` holds `ROOT_CA_BUNDLE`, the PEM bundle mbedTLS validates against. It must contain every certificate in each chain, not just the roots:

| Certificate                              | Needed for                           |
|------------------------------------------|--------------------------------------|
| WE1 (Google Trust Services) intermediate | Supabase                             |
| GlobalSign ECC Root CA R4                | Supabase                             |
| Sectigo Public Server Auth CA DV E36     | GitHub API — OTA release checks      |
| Sectigo Public Server Auth Root E46      | GitHub API                           |
| ISRG Root X1                             | GitHub's download CDN — OTA binaries |

**mbedTLS can't fetch missing intermediates (no AIA).** If a server omits one from its handshake and it isn't in the bundle, the connection fails with a misleading "connection refused". To capture a live chain when refreshing, in PowerShell:

```
$req = [Net.HttpWebRequest]::Create("https://<ref>.supabase.co")
$req.GetResponse() | Out-Null
$req.ServicePoint.Certificate
```

POWERSHELL

## 4.4 Flashing

- 1 Open `ClassAttendance_Current_RTC.ino`.
- 2 Board: **ESP32 Dev Module**. Select the COM port.
- 3 Upload.
- 4 Watch the Serial Monitor at 115200 baud to confirm it boots.

The current version is **1.10.0**. Bump `FIRMWARE_VERSION` on every release — OTA compares against it (section 7).

# 5 Onboarding an institution

## 5.1 Creating it

- 1 Go to **Create institution**.
- 2 Enter the **Name**, choose the **Type** — School, Office, or Shop / Retail — and set the **Timezone** (an IANA name such as *Africa/Accra*).
- 3 Choose which member types are tracked and each one's scan mode. All of this can be changed later in Settings.
- 4 Enter the first super admin's **Full name**, **Email**, and a **Temporary password** of at least 8 characters.
- 5 Press **Create**.

Everything else — labels, currency, branding, modules, holidays — the super admin sets up. Hand over the dashboard address, the credentials, and the right manuals (section 5.3).

## 5.2 Attaching scanners

Scanners only ever enter the system by powering on: a new one calls `/register` and waits, showing **PENDING** and breathing yellow.

- 1 Go to **Devices**. New scanners are listed under **Pending assignment**, by MAC address.
- 2 Press **Assign** and choose the institution. The scanner receives its own secret on its next poll and turns solid blue.
- 3 The institution's super admin then sees it under **Pending setup** and gives it a group and unit.

To take a scanner out of service, see section 6.1.

## 5.3 What to hand each client

The client PDFs are in `docs/for-clients/`.

| Type   | Give them                                                                                                                                                                      |
|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| School | <code>School_Manual.pdf</code> , <code>Super_Admin_Manual.pdf</code> , <code>Admin_Manual.pdf</code> , <code>Staff_Manual.pdf</code> , <code>Biometric_Privacy_Note.pdf</code> |
| Office | <code>Office_Manual.pdf</code> , <code>Super_Admin_Manual.pdf</code> , <code>Admin_Manual.pdf</code> , <code>Staff_Manual.pdf</code> , <code>Biometric_Privacy_Note.pdf</code> |
| Shop   | <code>Shop_Manual.pdf</code> , <code>Super_Admin_Manual.pdf</code> , <code>Admin_Manual.pdf</code> , <code>Biometric_Privacy_Note.pdf</code>                                   |

The type guide explains *what the system does for their kind of organisation*; the role guides explain *what each login can do*. The privacy note answers the question every school and data-protection officer asks about fingerprints.

## 5.4 Creating a super admin by hand

If you'd rather use SQL: create the user in Supabase Auth, copy the UUID, then

```
insert into profiles (id, role, institution_id)
values ('<uuid>', 'super_admin', '<institution-uuid>');
```

SQL

## 5.5 Institution lifecycle

| Status                   | Effect                                                                                                                   |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <code>active</code>      | Normal operation.                                                                                                        |
| <code>suspended</code>   | Temporary — e.g. non-payment. Every user is sent to <code>/suspended</code> . Toggled from the <b>Institutions</b> page. |
| <code>deactivated</code> | Long-term offboarding. Behaves like <code>suspended</code> ; data is kept for export. Set it in SQL.                     |

Suspension is enforced in the dashboard's `verifySession()`, not by RLS, because the dashboard reads through the service role. It doesn't touch scanners — they carry on and resume normally when the institution is reactivated.

**Deleting** an institution, from its page under **Institutions**, takes three confirmations. It removes all its people, attendance, periods, holidays, enrollment jobs, and accounts, and queues a wipe for each of its scanners. It can't be undone.

## 6 Secrets and scanner security

### 6.1 Per-scanner secrets

Each scanner's secret is minted by `assignment-poll` when it is first assigned, stored in `devices.device_secret`, and kept on the scanner in SPIFFS.

- **Block a scanner now:** set `devices.revoked = true`. Its next call is refused (401) and its screen shows **REVOKED**.
- **Wipe and re-provision it:** delete it from the **Devices** page (or insert a `device_resets` row). On its next poll it erases its identity and registers again as new, then receives a fresh secret when reassigned.

Prefer revoking over a hard delete when a scanner is lost or stolen: revocation is immediate, whereas the wipe only lands if the scanner ever connects again.

### 6.2 BOOTSTRAP\_SECRET

Compiled into the firmware and used only before a scanner is assigned. To rotate it, update `secrets.h` and the functions' `BOOTSTRAP_SECRET` together, then reflash or release over the air. Already-assigned scanners are unaffected; unassigned ones on the old value can't register until updated.

### 6.3 CRON\_SECRET

The cron job reads the secret from Vault each time it fires, so there's nothing to reschedule. Update the `cron_secret` entry in Vault and set the same value as the functions' `CRON_SECRET`, in that order and close together.

## 7 Over-the-air releases

Scanners check GitHub Releases each time they boot, after a random 0–120 second delay so a power cut doesn't send them all at once. They install only a strictly newer version.

- 1 In Arduino IDE: **Sketch** → **Export Compiled Binary**.
- 2 Create a GitHub release tagged `firmware-v<version>` — e.g. `firmware-v1.11.0` — higher than the current `FIRMWARE_VERSION`.
- 3 Attach the compiled `.bin`. The scanner installs the release's first `.bin` asset, so attach exactly one.
- 4 Scanners update on their next boot.

There's no forced push. During the update the screen shows **OTA UPDATE** and "Do not unplug", and the light breathes red.

## 8

## Database

- **Migrations** live in `backend/supabase/migrations/`. Apply new ones in the SQL editor, in filename order, after reading them. Never re-run old ones except when building a fresh database.
- **Backups:** Supabase takes daily backups on Pro and above, with point-in-time restore from its dashboard. For extra safety, take on-demand `pg_dump` exports with the connection string.
- **Raw data:** the SQL and table editors have full service-role access. There is no undo for destructive SQL.

## 9 Troubleshooting

| Symptom                                                 | Likely cause                                                                      | Fix                                                                                                                            |
|---------------------------------------------------------|-----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| "Bootstrap HTTP client error: connection refused"       | TLS chain failure                                                                 | Check <code>certs.h</code> has every intermediate (section 4.3). Try <code>setInsecure()</code> briefly to prove reachability. |
| 404 "Device not found" on the provisioning poll         | No <code>devices</code> row for that MAC, or no <code>provisioning_token</code>   | Check the table; confirm the provisioning-token migration ran.                                                                 |
| mark-absent: "schema net does not exist"                | pg_net not enabled                                                                | Enable it under Database → Extensions.                                                                                         |
| mark-absent returns 401                                 | Vault <code>cron_secret</code> and the functions' <code>CRON_SECRET</code> differ | Set them to the same value (section 6.3).                                                                                      |
| No absences appear at all                               | Cron job missing, or Vault secrets absent                                         | Check <code>select * from cron.job</code> and the Vault entries.                                                               |
| "unauthorized" after signing in                         | No <code>profiles</code> row, or the wrong role                                   | Insert or correct the profile for that auth user.                                                                              |
| Users sent to <code>/suspended</code>                   | <code>institutions.status</code> isn't <code>active</code>                        | Set it back to <code>active</code> if that wasn't intended.                                                                    |
| Scanner stuck on <b>PENDING</b>                         | Never assigned                                                                    | Assign it under <b>Devices</b> → <b>Pending assignment</b> .                                                                   |
| Scanner shows <b>REJECTED</b> or <b>REVOKED</b>         | Its secret is refused, or <code>revoked</code> is true                            | Check <code>devices.revoked</code> ; re-provision if needed.                                                                   |
| Idle screen says "No server"                            | WiFi works but HTTPS calls fail                                                   | Usually TLS or DNS. It re-associates every 60 s and reboots after 5 minutes unreachable; scans stay queued.                    |
| Re-registers after every SPIFFS clear                   | The clear ran after <code>loadProvisioning()</code> cached the id                 | The clear must be the first thing in <code>setup()</code> , followed by <code>ESP.restart()</code> .                           |
| OTA fails fetching GitHub releases                      | ISRG Root X1 or a Sectigo certificate missing                                     | Ensure all five certificates are in <code>ROOT_CA_BUNDLE</code> .                                                              |
| Enrollment jobs fail as <b>OCCUPIED</b> after promotion | Old prints still on the sensor                                                    | The job's <b>+ overwrite</b> action, or a <b>Clear all</b> job on that scanner.                                                |

## 10 Pre-handover checklist

Run through this before you leave a client's site.

- ☐ Institution created with the right **type** and **timezone**

- ☐ Super admin signed in and changed the temporary password
- ☐ Settings done: labels, scan modes, days tracked, punctuality, time display, branding (and currency and modules for a shop)
- ☐ Current period active, and its holidays or closed days entered
- ☐ Every scanner assigned, configured with a group and unit, and showing solid blue
- ☐ **A master finger enrolled on every scanner** — and the client knows whose it is
- ☐ The client has enrolled at least one person themselves, unaided
- ☐ A real scan recorded and found on the dashboard
- ☐ For shops: catalog loaded, a test sale recorded, and a cashier login working
- ☐ Manuals and the privacy note handed over (section 5.3)
- ☐ The client knows how to reach you